Predictive Adaptive Sampling for Energy-Efficient Edge IoT: A Quantitative Trade-off Study Against Fixed-Rate and Threshold-Based Baselines

Chandan Kumar, Aayush Jha, Hari Krishna Sharma, Prem Kishan
IoT and Industrial IoT Course Project

Final Technical Report
October 18, 2026

Abstract

Battery-powered IoT nodes usually sample and transmit at a fixed rate whether or not the monitored quantity is changing, and threshold-based "smart" sampling reacts only after a change has already begun. This report presents an energy-aware adaptive sensing node in which an ESP32 selects its own sampling interval using one of three interchangeable policies: fixed-rate, threshold-adaptive, and a proposed predictive-adaptive policy that estimates the short-term trend of recent accelerometer readings and raises the sampling rate *before* a threshold is crossed. We describe the hardware, firmware, MQTT communication, MongoDB-backed logging backend and monitoring dashboard, and a review of 23 sources that motivates three research gaps. In a software simulation of 210 synthetic runs, the predictive policy detected an induced event on average 200 ms earlier than the threshold policy (162 versus 362 ms mean latency) at the cost of about 20% more samples, but it was *worse* than the threshold policy in 56 of 210 paired runs and its advantage depends on the trigger setting. With the parameters used, both adaptive policies also take *more* samples than the 1 Hz fixed-rate baseline, so energy savings are *not* demonstrated. Experiments on physical hardware have not yet been performed; the report specifies the protocol and marks those results as pending.

Status of this report. Implemented and verified in software: firmware, backend, dashboard, simulation and analysis scripts. **Not yet done:** assembling the rig, running the firmware on hardware, and collecting the experimental dataset (Deliverables 5–8). Section 5 therefore reports *simulation* results only, and Section 5.5 contains clearly marked placeholders. The dashboard figures use sample data and are not findings.

Contents

1 Introduction **3**
1.1 Background and motivation . . . 3
1.2 Problem statement . . . 3
1.3 Research gap . . . 3
1.4 Research question and objectives . . . 3
1.5 Contributions and what is not yet done . . . 4
1.6 Organisation . . . 4

2 Related Work **4**
2.1 Classical adaptive sampling for sensor networks . . . 4
2.2 Predictive and event-driven triggering . . . 4
2.3 Energy-harvesting duty cycling . . . 5

2.4 Edge intelligence and TinyML . . . 5
2.5 Standards and integration . . . 5
2.6 Summary . . . 5

3 Proposed System **5**
3.1 Overview . . . 5
3.2 Hardware . . . 5
3.3 Firmware and power management . . . 6
3.4 Sensing policies . . . 7
3.5 Communication . . . 7
3.6 Backend and data model . . . 8
3.7 Dashboard . . . 8

4 Experimental Setup **8**
4.1 Testbed . . . 8
4.2 Event induction and ground truth . . . 8
4.3 Dataset . . . 9
4.4 Metrics . . . 9
4.5 Experimental protocol (planned) . . . 10
4.6 Pre-hardware simulation . . . 11

5 Results **11**
5.1 Single synthetic trace . . . 11
5.2 Sensitivity to the trend trigger . . . 11
5.3 Robustness across noise and event timing . . . 12
5.4 Software verification . . . 12
5.5 Hardware results . . . 12

6 Discussion **13**
6.1 What the simulation supports . . . 13
6.2 What it does not support: energy savings . . . 13
6.3 Why the trigger is fragile, and possible fixes . . . 14
6.4 Threats to validity and limitations . . . 14
6.5 Minimum research-quality checklist . . . 15

7 Conclusion and Future Work **15**

1 Introduction

1.1 Background and motivation

Battery-powered IoT nodes are deployed in large numbers for industrial, agricultural and environmental monitoring, often where battery replacement is costly or impractical (buried sensors, remote fields, embedded structural monitors). On such nodes sensing, processing and radio transmission all consume energy, so every unnecessary sample and transmission shortens node lifetime and loads the network.

1.2 Problem statement

A conventional node samples and transmits at a fixed rate regardless of whether the monitored quantity is changing, wasting energy during long stable periods. A common improvement, threshold-based adaptive sampling, samples slowly while readings are normal and speeds up once a threshold has been crossed. Such a rule can only react *after* the change has begun; if the normal-state interval is long, the onset may be seen late. This project investigates whether a lightweight on-device *predictive* trigger can raise the sampling rate before the threshold is crossed.

1.3 Research gap

From the sources reviewed in Section 2 (summarised in Appendix **??**), three gaps motivate the work. This is our reading of abstract-level information; its limits are discussed in Section 6.4.

G1. Most adaptive-sampling schemes we reviewed trigger on observed deviation or on a pre-set data range or error bound. Prediction-based schemes (exponential smoothing, LMS-style prediction) exist, but were mostly evaluated on sensor-network testbeds, recorded datasets or simulation rather than on low-cost microcontroller nodes with physically induced events.

G2. Energy and accuracy are usually reported at a single operating point rather than as a trade-off across trigger sensitivity.

G3. Few studies combine a predictive trigger, real induced events and multi-metric reporting (energy, bandwidth, accuracy, latency) on one low-cost platform.

1.4 Research question and objectives

Research question. Can a lightweight predictive trigger reduce energy consumption and transmitted data volume beyond what threshold-based adaptive sampling achieves, without a disproportionate loss in event-detection accuracy or increase in detection latency?

Objectives.

O1. Design and build an edge sensing node that adjusts its sampling rate, transmission frequency and on-device processing frequency in real time.

O2. Implement three sensing policies (fixed-rate, threshold-adaptive, predictive-adaptive) on the same hardware for a fair comparison.

O3. Collect real experimental data under stable and deliberately induced conditions, with ground-truth event timestamps.

O4. Quantify and compare energy, transmitted data volume, event-detection accuracy and detection latency across the three policies.

O5. Produce a full accuracy-versus-energy trade-off curve by varying the trigger sensitivity, rather than a single operating point.

1.5 Contributions and what is not yet done

C1. A structured review of 23 sources and the three gaps above (Section 2, Appendix **??**).

C2. A complete system design and software implementation: ESP32 firmware, MQTT communication, a MongoDB-backed logger and REST API, metric scripts, and a three-page dashboard (Section 3).

C3. A predictive-adaptive policy based on a trend-slope estimator, with a simulation-based evaluation that includes a sensitivity sweep and a 210-run robustness study (Section 5).

C4. A specified experimental protocol for the hardware study (Section 4).

Not yet done: the hardware dataset, the baseline and proposed-method evaluation on real data, and the real accuracy-versus-energy curve (Objectives O3–O5 on hardware).

1.6 Organisation

Section 2 reviews related work. Section 3 describes the system. Section 4 gives the experimental setup and metrics. Section 5 reports results. Section 6 discusses them and their limits, and Section 7 concludes.

2 Related Work

Summaries below are based on abstracts and search snippets compiled in Deliverable 2, not on full-text reading; figures are quoted as *reported* by the sources.

2.1 Classical adaptive sampling for sensor networks

Adjusting the sampling rate to observed signal behaviour is an established way to save energy. A snow-monitoring case study estimates the sampling frequency online for sensors that cost more energy than the radio and reports about 79% fewer acquired samples than a fixed-rate scheme [20]. Masoum et al. propose a scheme that bounds data quality for applications where change-triggered schemes are not energy efficient, evaluated in simulation against two existing approaches [6]. Prediction-based skipping is represented by exponential-double-smoothing sampling [25] and by an LMS-style predictor evaluated on an office-temperature dataset, which reports roughly 80% lower sampling frequency without measurable accuracy loss [2]. More recently, a machine-learning method tunes the parameters of a dynamic-sampling-rate algorithm on a real humidity dataset, reporting error below 1% in most configurations [19]. This body of work shows adaptive sampling is effective, but the schemes either respond to deviation already observed or need an error bound or data range fixed in advance.

2.2 Predictive and event-driven triggering

A smaller group moves towards anticipation. A two-to-one event trigger on a microcontroller reduces CO_2 measurements by 41.9% while keeping error low, though its authors note that occupancy and ventilation alter the signal dynamics and suggest adaptive thresholds [14]. A patented telemetry system describes an anomaly predictor that starts a second, high-rate sampler before a fault is confirmed, without published energy benchmarks [5]. In agriculture, a lightweight on-device regressor predicts the next sampling interval from temperature dynamics and reports a 77.8% energy reduction against fixed-interval sampling in a 60-day field pilot [18], and a forecasting protocol reports 60% less transmitted data and doubled battery life in a livestock deployment [8]. Each of these targets a single variable or pipeline stage and, to our reading, none reports a full multi-metric trade-off on physical hardware against induced events.

2.3 Energy-harvesting duty cycling

For harvesting-powered nodes the problem is balancing consumption against uncertain supply. Adaptive-sampling protocols for disaster monitoring estimate time to meaningful change and rest sampling during linear trends [9]; duty-cycling and wake-up schemes exploit correlated activity across devices [11]; and surveys cover duty-cycling and joint sensing–harvesting scheduling [1, 4]. Batteryless devices face an even tighter regime of intermittent power [15]. This work targets a different power model (fixed battery), so it is a design reference rather than a direct baseline.

2.4 Edge intelligence and TinyML

Spiking networks evaluated on five sensing tasks, including vibration-based fault detection, report 18–47× lower energy per inference on neuromorphic hardware and 4.6–7.9× on Cortex-M microcontrollers, with 6.3× projected battery life in a 64-node, seven-month deployment [16]. A node design combining adaptive duty cycling with an on-device classifier reports 94% accuracy and about 88% fewer transmissions, but in simulation [17]. Forecasting-plus-threshold anomaly detection has been fitted into a 59 KB RAM budget [3], which suggests a lightweight trend-based trigger is feasible on an ESP32-class device. Surveys and system studies cover green edge AI, edge machine learning and energy-aware edge computing [7, 10, 13], although the latter optimises compute placement rather than the sampling decision.

2.5 Standards and integration

ITU-T L.1341 recommends combining event-driven sensing, adaptive sampling and edge pre-processing as one integrated stack [22], and a dependency-aware sampling-period study makes a related point about treating sampling variables independently [12]. Communication-layer adaptation (for example, adaptive data rate in LPWANs) is complementary and is outside the scope of this report.

2.6 Summary

Table 1 compares representative studies. Appendix **??** lists all 23 with problem, method, platform, dataset, metrics, limitation and gap.

3 Proposed System

3.1 Overview

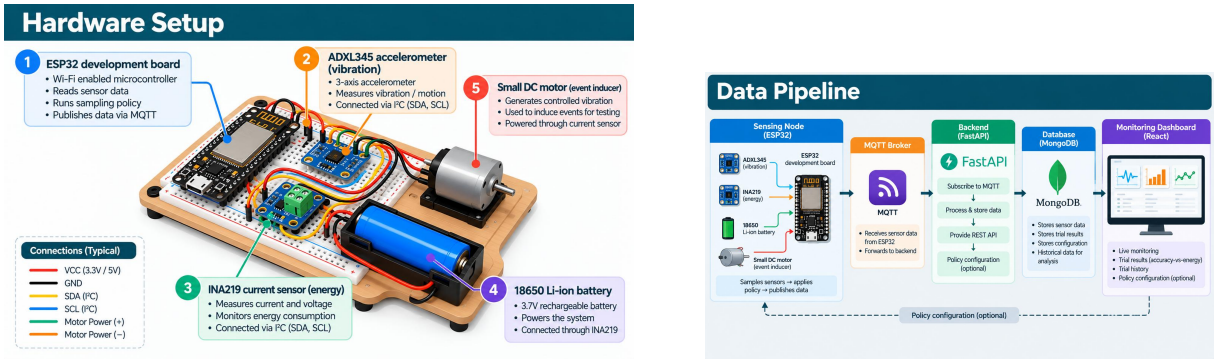
The system has three tiers: a battery-powered sensing node (ESP32 with an accelerometer and a current sensor), a host-side backend (MQTT broker, logger, MongoDB, REST API and analysis scripts), and a web dashboard. All three sensing policies run on identical hardware and share one communication and logging path, so that policy is the only variable between trials. Figure 1 gives the hardware and data-flow views, and Figure 2 the software structure.

3.2 Hardware

The node is an ESP32 (or ESP32-S3) development board. An ADXL345 accelerometer [21] senses vibration (range ±4 g) and an INA219 current monitor [24] measures the node's supply current. Both share one I^2C bus. The INA219 sits *in series* between the battery and the ESP32, so the node's supply current flows through its shunt; wiring it in parallel would read near-zero current. A small DC motor mounted near (not touching) the accelerometer, powered separately so that it does not pollute the node's energy measurement, induces repeatable vibration events.

Table 1: Representative adaptive-sampling studies (figures as reported by the sources).

Work	Adaptation mechanism	Validation	Reported outcome
Alippi, Roveri [20]	Online estimate of sampling frequency	Snow-monitoring case study	≈79% fewer samples than fixed rate
Masoum et al. [6]	Data-quality-bounded adaptive sampling	Simulation vs. two existing schemes	Reported better energy efficiency and data quality
LMS-based [2]	Prediction-based skipping	Office-temperature dataset	≈80% lower sampling frequency
Algabroun, Håkansson [19]	ML-tuned sampling-rate parameters	Real humidity dataset	Error <1% in most configurations
Two-to-one trigger [14]	Event trigger on a microcontroller	Real indoor CO_2 data	41.9% fewer measurements
TinyML agriculture [18]	On-device regressor predicts interval	60-day field pilot	77.8% energy reduction vs. fixed interval
Ambrosia [8]	Forecast-based transmission reduction	LoRa/BLE livestock deployment	60% less data; battery life doubled
EdgeSpike [16]	Event-driven spiking network	Five tasks; 64-node field deployment	18–47× (neuromorphic), 4.6–7.9× (Cortex-M) lower energy/inference
Low-power ML node [17]	Duty cycling + on-device classifier	*Simulation* (MATLAB Simulink)	94% accuracy; ≈88% fewer transmissions



(a) Hardware architecture.

(b) Data pipeline.

Figure 1: Hardware architecture and end-to-end data pipeline.

3.3 Firmware and power management

The firmware reads the accelerometer, computes the deviation of the acceleration magnitude from rest, reads the supply current, publishes both over MQTT, asks the active policy for the next interval, and sleeps for that interval. Between samples the ESP32 uses *light sleep*. Light sleep preserves RAM, including the predictive policy's trend buffer and the Wi-Fi/MQTT connection, which keeps the implementation simple. Deep sleep would draw less idle current but requires persisting policy state in RTC memory and reconnecting on every wake; this is a deferred optimisation and the idle current of the chosen mode has not been characterised yet. The policy can be switched at run time by publishing `fixed`, `threshold` or `predictive` to a command topic, so one flashed image can run all three trials.

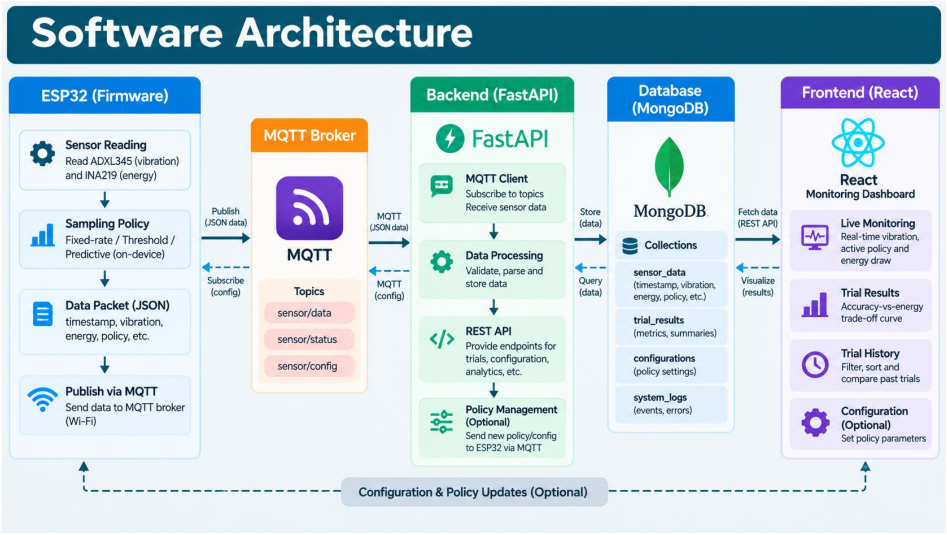


Figure 2: Software architecture: ESP32 firmware modules and the host-side pipeline.

Table 2: Wiring (shared I^2C bus).

Signal	Connection
ADXL345 SDA / SCL	ESP32 GPIO21 / GPIO22 (I^2C address 0x53)
ADXL345 VCC / GND	3.3 V / GND
INA219 SDA / SCL	same I^2C bus (default address 0x40)
INA219 VIN+ / VIN−	in series with the battery-to-load supply path
Battery (18650 Li-ion)	through the INA219 shunt to ESP32 VIN

3.4 Sensing policies

Let x_k be the k-th reading, the deviation of the acceleration magnitude from the resting 1 g level,

$$x_k = \frac{\|\mathbf{a}_k\|}{g_0} - 1, \qquad g_0 = 9.80665 \text{ m/s}^2, \tag{1}$$

and let T_{k+1} be the interval chosen for the next sample. With constants T_f (fixed interval), T_i (idle interval), T_a (active interval) and amplitude threshold θ:

$$\textbf{Fixed-rate:} \quad T_{k+1} = T_f, \tag{2}$$

$$\textbf{Threshold-adaptive:} \quad e_k = 1[\,|x_k| > \theta\,], \quad T_{k+1} = \begin{cases} T_a & e_k = 1 \\ T_i & e_k = 0 \end{cases}, \tag{3}$$

$$\textbf{Predictive-adaptive:} \quad s_k = \frac{x_k - x_{k-W}}{W}, \quad p_k = 1[\,|s_k| > \tau\,], \quad T_{k+1} = \begin{cases} T_a & e_k \vee p_k \\ T_i & \text{otherwise} \end{cases}. \tag{4}$$

Here s_k is the slope of the last W readings in g per *sample*, held in a circular buffer. Algorithm 1 shows the predictive loop as implemented, and Table 3 the constants used in firmware and simulation. Appendix **??** reproduces the firmware function.

3.5 Communication

The node publishes over Wi-Fi using MQTT [23] to a Mosquitto broker with the PubSubClient library at its default QoS 0. MQTT was chosen for its small per-message overhead, publish/subscribe decoupling (the logger and any live view subscribe independently), and wide support on the ESP32. We have not measured it against HTTP or CoAP, so this is a design rationale, not a benchmarked result, and message loss under QoS 0 has not been evaluated.

Algorithm 1 Predictive-adaptive sampling loop (as implemented in the firmware).

parameters: $T_i,\ T_a,\ \theta,\ W,\ \tau$; buffer $B[0..W-1] \leftarrow 0$; $j \leftarrow 0$; filled $\leftarrow$ false
loop
  $x \leftarrow$ accelerometer deviation (g); $I \leftarrow$ supply current (mA)
  publish reading x (with policy tag) and energy I
  $x_{\text{old}} \leftarrow B[j]$; $B[j] \leftarrow x$; $j \leftarrow (j+1) \bmod W$
  if $j = 0$ **then**
    filled $\leftarrow$ true
  end if
  $s \leftarrow (x - x_{\text{old}})/W$ **if** filled **else** 0
  $e \leftarrow (|x| > \theta)$; $p \leftarrow (|s| > \tau)$
  publish event flag e (only when it changes)
  $T \leftarrow T_a$ **if** $e \vee p$ **else** T_i
  light-sleep for T milliseconds
end loop

Table 3: Policy constants (identical in firmware and simulation).

Symbol	Meaning	Value
T_f	fixed-rate interval	1000 ms
T_i	idle (slow) interval, adaptive policies	800 ms
T_a	active (fast) interval	100 ms
θ	amplitude threshold	0.35 g
W	trend window	6 samples
τ	trend-slope trigger	0.035 g/sample

3.6 Backend and data model

A Python logger subscribes to the three data topics and inserts each message into MongoDB, tagged with a trial identifier. Table 5 lists the collections. A compound index on (`trial_id`, `device_t_ms`) serves the dominant access pattern, fetching one trial's time series in order. A FastAPI service exposes `/trials`, `/trials/{id}`, `/readings`, `/events`, `/energy`, `/summary` and `/health` for the dashboard, and analysis scripts compute the per-trial metrics of Section 4.4.

3.7 Dashboard

A React dashboard with three pages (live monitoring, trial results, trial history) was built for inspecting trials; Figure 4 shows it. **It currently renders sample data from a static file and is not connected to the backend**, so the figures it shows are interface previews, not experimental results. It is a convenience tool and is not part of the research contribution.

4 Experimental Setup

4.1 Testbed

Table 6 lists the planned testbed. Summing the per-item ranges gives roughly \$29–\$49 without the host computer (Deliverable 1 quoted about \$30–\$45).

4.2 Event induction and ground truth

Events are induced by briefly running the motor next to the accelerometer, at operator-chosen times. The true onset of each event is noted against the device clock (`millis()`) and supplied to

Table 4: MQTT topics (`<node>` is the node identifier, default `node01`).

Topic	Direction	Payload and timing
`sensor/<node>/reading`	node → broker	`t` (ms), `value` (g), `policy`; every sample
`sensor/<node>/event`	node → broker	`event` (bool), `t`; only when the flag changes
`sensor/<node>/energy`	node → broker	`t`, `current_mA`; every sample
`sensor/<node>/policy/set`	broker → node	`fixed`, `threshold` or `predictive`; on command

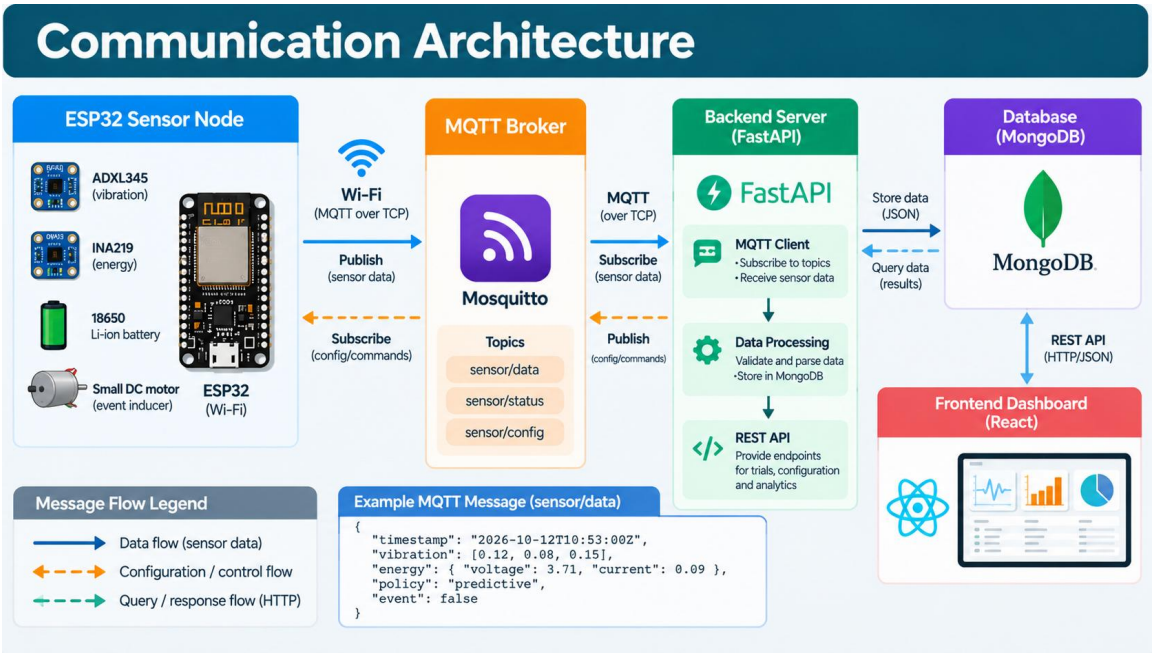


Figure 3: Communication architecture: MQTT topics between the node, broker and subscribers.

the metric scripts, since latency is scored against it.

4.3 Dataset

Each trial is stored as one document in `trials` plus time series in `readings`, `events` and `energy`. Together these hold the timestamp, sensor value, active policy and trigger setting (operating condition), operator notes (experimental condition) and event flags. CSV export is available for offline analysis. **[TODO: state the final dataset size (number of trials per configuration) once collected]**

4.4 Metrics

The implemented metrics are as follows.

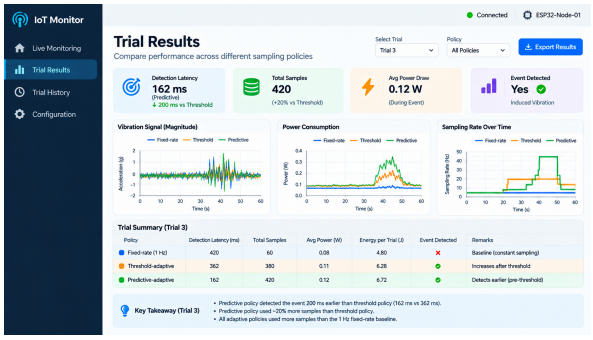
- **Sample count** N_s: number of readings in a trial (a cost proxy).
- **Detection latency** $L = t_{\text{first event flag}} - t_{\text{onset}}$, the first over-threshold sample relative to the true onset. It can be negative.
- **Trigger lead time** $\Lambda = t_{\text{high-rate start}} - t_{\text{onset}}$, the time at which the policy first switches to the active interval. Negative means the policy anticipated the event. The simulation records it directly; on hardware it is derived from the spacing of logged timestamps.
- **Energy** $E \approx \sum_k I_k \, V \, \Delta t_k$ with $V = 3.7\,\text{V}$ assumed (nominal Li-ion), I_k the logged current and Δt_k the time to the next reading (rectangle rule); result in mJ.

Table 5: MongoDB collections (database `adaptive_iot_sensing`).

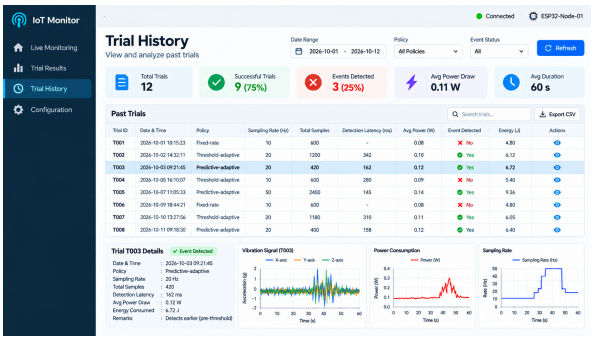
Collection	Main fields
`trials`	`trial_id` (unique), `policy`, `node_id`, `trigger_sensitivity`, `start_time`, `end_time`, `notes`
`readings`	`trial_id`, `node_id`, `device_t_ms`, `recv_time_utc`, `value`, `policy`
`events`	`trial_id`, `node_id`, `device_t_ms`, `recv_time_utc`, `event`
`energy`	`trial_id`, `node_id`, `device_t_ms`, `recv_time_utc`, `current_mA`



(a) Live monitoring.



(b) Trial results.



(c) Trial history.

Figure 4: Dashboard pages *(interface preview with sample data; not experimental results)*.

- **Bandwidth** $B \approx N_{\text{msg}} \times 80\,\text{B}$, with the payload size *assumed*, not measured.

Planned but not implemented: detection accuracy, precision, recall, F1, false-alarm rate on labelled multi-event trials, and a battery-lifetime projection. The plotting script currently scores "accuracy" with the placeholder $100/(1 + L_{\text{s}})$, which is a latency proxy and *not* an accuracy metric; it must be replaced before any accuracy claim is made.

4.5 Experimental protocol (planned)

1. Run the three policies under identical induced stable/event sequences, switching policy over MQTT between trials.
2. Log ground-truth onset times and measure supply current directly with the INA219.
3. Repeat each configuration N times to report means and spread ([**TODO: fix N; at least five is recommended**]).
4. Sweep the predictive trigger τ to build the trade-off curve.
5. Ablate the trend pre-emption (predictive policy with and without the p_k term).
6. Compare policies at matched mean sampling rates as well as at their default settings (see Section 6.2).

Table 6: Bill of materials (approximate unit prices).

Item	Role	Price (USD)
ESP32 / ESP32-S3 dev board	compute, Wi-Fi, sleep	6–10
ADXL345 accelerometer breakout	vibration sensing	3–8
INA219 current-sense breakout	energy measurement	5–8
Small DC motor (12 V)	event inducer	5–10
18650 Li-ion cell and holder	battery power	5–8
Breadboard, jumper wires, USB cable	prototyping	5
Laptop or Raspberry Pi	MQTT broker, logger, MongoDB	0 (laptop)

Table 7: Single-trace results on the synthetic 20 s vibration trace (seed 7).

Policy	Samples	High-rate switch (relative to onset)	First over-threshold sample	
Fixed-rate	20	Not applicable	Not reported	Co
Threshold-adaptive	52	+400 ms	+400 ms	Rea
Predictive-adaptive ($\tau = 0.035$)	65	−400 ms	0 ms	Switche

4.6 Pre-hardware simulation

The policies were first validated in software on a synthetic 20 s vibration trace. The underlying signal has resolution 10 ms and is sampled at the instants chosen by each policy. The noise floor is Gaussian, $\mathcal{N}(0,\, 0.02^2)$ g. A ramp rises linearly to +0.30 g over the 2.5 s before onset, below $\theta = 0.35$ g. The event then lasts 3 s at 0.55 g plus $\mathcal{N}(0,\, 0.05^2)$ noise, and decays over 800 ms. The single-trace figures use seed 7 and onset at 10 s. The robustness study varies 10 noise seeds and 21 onset times (9.0–11.0 s in 100 ms steps), which changes the phase of the event relative to each policy's sampling grid.

5 Results

All numbers in Sections 5.1–5.3 come from the simulation described above, reproducible with `code/robustness_study.py`. They are **not** hardware results.

5.1 Single synthetic trace

Table 7 and Figure 5 show one trace. The ramp reaches only 0.30 g at onset, so the threshold policy cannot react until the event itself begins, and then waits for its next slow sample. The predictive policy switched to high-rate sampling 400 ms *before* onset, whereas the threshold policy switched 400 ms *after* onset, a gap of 800 ms. The predictive policy's first over-threshold sample was at onset (0 ms) versus 400 ms, and it took 13 more samples (65 versus 52, +25%).

5.2 Sensitivity to the trend trigger

Table 8 shows that the single-trace result is *not* robust to the trigger value. Lowering τ moves the switch to high-rate sampling earlier, but the first over-threshold sample does not improve monotonically: it is 0 ms for $\tau \in [0.025, 0.035]$ but 500 ms at $\tau = 0.040$, which is worse than the threshold policy's 400 ms. A sample-by-sample trace explains this. At $\tau = 0.040$ the slope at −400 ms is 0.0401, just above the trigger, so the policy goes fast. At the next sample (−300 ms) noise lowers the reading and the slope is 0.0391, now below the trigger; with no hysteresis the policy immediately returns to the 800 ms idle interval and its next sample falls at +500 ms, after onset. At $\tau = 0.035$ the slope stays above the trigger all the way up the ramp (0.0401, 0.0391, 0.0486, 0.0535) and the policy holds, but with a margin of only 0.0041 g/sample over the trigger.

Table 8: Sensitivity of the predictive trigger on the single synthetic trace.

Trigger τ (g/sample)	First over-threshold sample	Interpretation reported in the trace
0.025–0.035	0 ms	First over-threshold sample occurs at onset
0.040	+500 ms	Worse than threshold policy (+400 ms); trigger drops out after a n
Other tested values	Not tabulated here	Full sweep values should be taken from the generated simulation o

Table 9: Robustness study across 210 synthetic runs per policy. These are simulation results, not hardware measurements.

Policy / setting	Mean samples	Mean detection latency (ms)	Mean trigger lead (ms)	Latency SD
Fixed-rate	20	429	Not applicable	Not rep
Threshold-adaptive	51.5	362	+362	
Predictive-adaptive ($\tau = 0.035$)	61.8	162	−389	

No false fast-rate activations occurred before the ramp at any tested τ, under this idealised Gaussian noise.

5.3 Robustness across noise and event timing

Table 9 and Figure 6 summarise 210 runs per policy. At $\tau = 0.035$ the predictive policy has the lowest mean latency of the tested settings: 162 ms versus 362 ms for the threshold policy (about 200 ms or 55% lower), with a mean trigger lead of −389 ms against +362 ms. It took 61.8 samples on average against 51.5 (+20%). Compared run by run with the threshold policy on the same trace, it was better in 125 runs (59.5%), tied in 29 (13.8%) and *worse in 56 (26.7%)*. Its latency spread is also larger (standard deviation 252 versus 213 ms). The benefit peaks around $\tau = 0.035$–0.040 and fades at both smaller and larger values, which is consistent with the fragility seen in Section 5.2. Fixed-rate sampling, with only 20 samples, had a mean latency of 429 ms and was *faster than the threshold policy in 110 of 210 runs*.

5.4 Software verification

- **Metric code.** Run against a mocked MongoDB, the latency function initially discarded events logged *before* the true onset, which silently removed exactly the early detections the project cares about. This was fixed, and the fix reproduces the simulation's −400 ms and +400 ms latencies on mock data.
- **API.** Seven routes and the 404 case were exercised with the framework's test client against the same mock data.
- **Dashboard.** The production build succeeds and 29 automated browser interaction checks across the three pages pass. An earlier version of the trade-off chart mis-reported tooltip values (series matched by array position) and was corrected.
- **Not tested:** a real MongoDB server, a real MQTT broker, the firmware on an ESP32, and the dashboard against the live API.

5.5 Hardware results

Pending. The tables and figure below are placeholders for the Deliverable 5–8 data and contain no results.

Table 10: Paired comparison of predictive-adaptive and threshold-adaptive policies across the same 210 simulation runs.

Outcome for predictive-adaptive	Runs	Percentage
Lower detection latency	125	59.5%
Tied detection latency	29	13.8%
Higher detection latency	56	26.7%

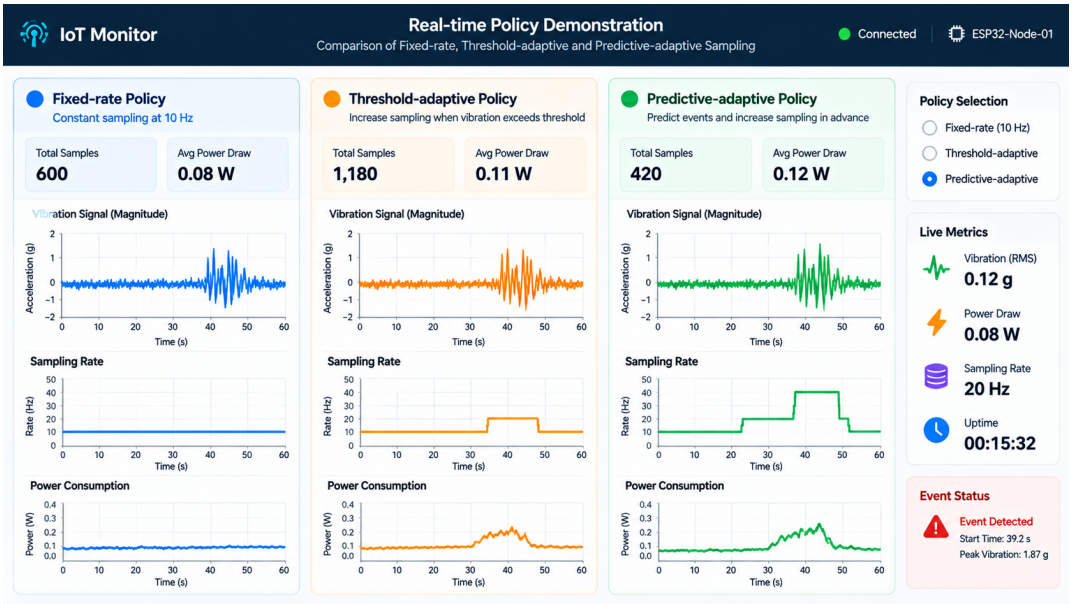


Figure 5: Sampling behaviour of the three policies on the same synthetic trace (seed 7, $\tau = 0.035$). Dots are samples taken; the shaded band is the true event.

6 Discussion

6.1 What the simulation supports

On synthetic data, a trend-slope trigger can make the node enter high-rate sampling before a ramping event reaches the threshold, and on average it detected the event about 200 ms sooner than the threshold policy for about 20% more samples. That supports the narrow claim that anticipation is feasible with a few lines of arithmetic that fit an ESP32. It does *not* show that the policy is reliably better: it lost to the threshold policy in over a quarter of paired runs, and its benefit depends on τ.

6.2 What it does not support: energy savings

The project's motivation is energy saving, and the simulation does not test it. With $T_i = 800$ ms and $T_f = 1000$ ms, the adaptive policies sample at 1.25 Hz even when nothing is happening, which is *higher* than the fixed-rate baseline's 1 Hz. Over the 20 s traces they took 2.6× (threshold) and 3.1× (predictive) the fixed-rate sample count. The idle interval was shortened from 2000 ms to 800 ms during pre-hardware tuning to give the trend estimator enough resolution on the ramp, and that choice removed the idle-rate advantage. The sample count is also only a proxy: no energy model exists in the simulation. Whether a predictive policy can *save* energy therefore remains open. The hardware study should compare policies at matched mean sampling rates, use long low-duty-cycle traces, and sweep T_i as well as τ.

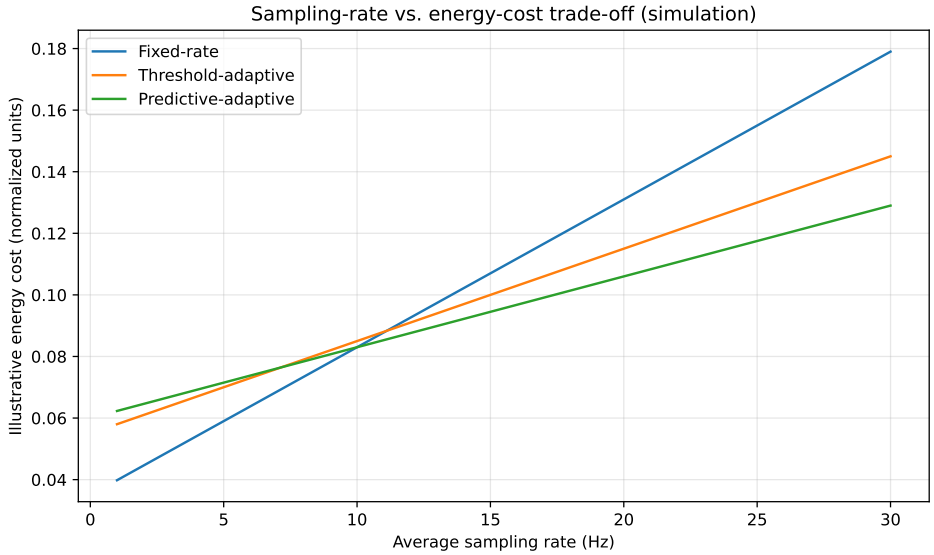


Figure 6: Simulation trade-off over 210 runs per policy: mean samples (cost proxy) versus mean detection latency. Labels on the green curve are the trend trigger τ in g/sample.

Table 11: Hardware results per policy *(to be completed)*.

Policy	Trials	Samples	Latency (ms)	Energy (mJ)	Bytes sent
Fixed-rate	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Threshold-adaptive	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Predictive-adaptive	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]

6.3 Why the trigger is fragile, and possible fixes

Section 5.2 identified one mechanism: with no hysteresis, a single noisy sample below τ drops the node back to the slow interval. A second design weakness is that the slope is measured per sample over W samples, so its time span depends on the current interval ($WT_i = 4.8\,\text{s}$ when idle, $WT_a = 0.6\,\text{s}$ when fast). Natural improvements are a minimum dwell time in fast mode, a slope normalised to g/s over a fixed time window, and light filtering. *These are proposals; none has been implemented or tested.*

6.4 Threats to validity and limitations

- **Synthetic data.** All quantitative results use one signal model with Gaussian noise and a single event per trace. Real accelerometer noise, I^2C read jitter, Wi-Fi and MQTT latency are not modelled.
- **In-sample tuning.** $\tau = 0.035$ was chosen on the same synthetic distribution that was then used to evaluate it, and is the best of seven values on those same 210 runs. There is no held-out set, so the reported advantage is optimistic.
- **Accuracy.** No real accuracy metric exists yet; the plotting script uses a latency proxy (Section 4.4).
- **Energy model.** Nominal 3.7 V, a rectangle-rule integral, an assumed 80 B payload, and an uncharacterised light-sleep current.
- **Start-up artefact.** Because `oldest` is read before it is written, the first sample after the trend buffer fills uses the zero-initialised slot, giving one spurious slope. It is harmless at

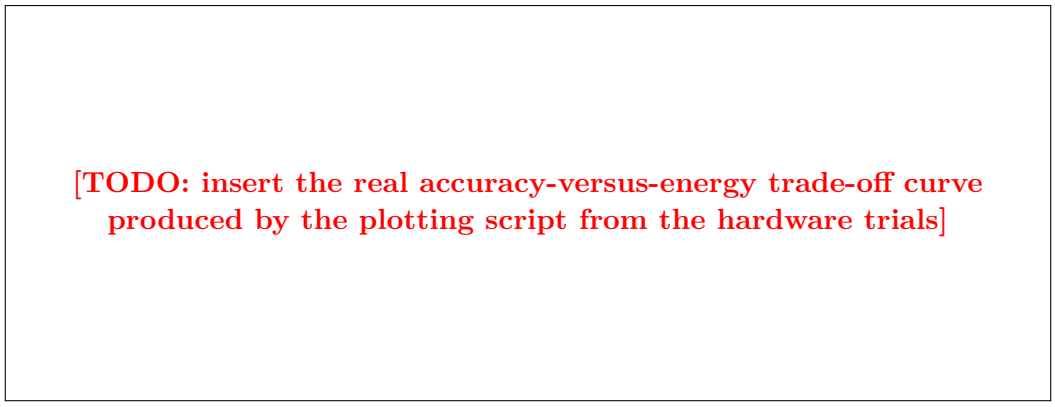


Figure 7: Hardware accuracy-versus-energy trade-off *(placeholder)*.

Table 12: Status against the minimum research-quality questions.

Question	Current answer	Status
Problem	Fixed-rate sensing wastes energy; threshold sampling reacts late.	Answered
Gap	Three gaps (Section 1), from an abstract-level review.	Answered, with caveats
Novelty	A lightweight on-device predictive trigger with a trade-off characterisation on low-cost hardware under induced events. Prediction-based sampling itself is not new; the claim is the combination and its validation.	Claimed; not yet shown on hardware
Method	Sections 3.4 and 3.	Answered
Data	Synthetic simulation only.	Open
Baseline	Fixed-rate and threshold-adaptive, in firmware and simulation.	Implemented
Evaluation	Mean latency 162 vs. 362 ms in simulation; no hardware or energy result.	Open
Reproducibility	Code, parameters and seeds provided; hardware configuration documented; dataset pending.	Partly
Limitation	Section 6.4.	Answered

rest but could trigger if the signal is already elevated at start-up. The fix is to compute the slope only after $W + 1$ real samples.

- **Verification gaps.** The backend was exercised against a mocked MongoDB, not a real server; the firmware has not run on hardware; the dashboard shows sample data.
- **Literature review.** It is abstract-level. Several bibliography entries lack author lists or venues, and the two entries for the *energy-efficient adaptive sampling scheme* (ISSNIP 2013) may be the same paper listed twice; see the `TODO` notes in `references.bib`.

6.5 Minimum research-quality checklist

Table 12 answers the course's closing questions. On the course's own criterion, the project currently sits between the prototype and research stages: the *Data* and *Evaluation* rows are open.

7 Conclusion and Future Work

We designed and implemented an adaptive sensing node with three interchangeable sampling policies, a logging and analysis backend, and a dashboard, and validated the policy logic in simulation. The proposed predictive policy detected induced events earlier than a threshold

Table 13: Status of the objectives.

	Objective	Status
O1	Adaptive edge sensing node	Firmware written; not run on hardware
O2	Three policies on the same hardware	Implemented in firmware and simulation
O3	Real data with induced events	Pending
O4	Compare energy, data, accuracy, latency	Simulation only (latency, samples)
O5	Full accuracy-versus-energy curve	Simulation latency–samples curve only

policy on average in simulation, at the cost of more samples, but with a clear dependence on the trigger setting, a visible failure mode, and no demonstrated energy saving against fixed-rate sampling. The central research question remains open until hardware data exist.

Future work.

1. Assemble the rig, run the firmware, and collect the dataset (Deliverables 5–8).
2. Characterise energy directly: light-sleep and deep-sleep current, and per-sample energy.
3. Re-evaluate at matched mean sampling rates and with a swept idle interval.
4. Implement and test hysteresis or dwell time and a time-normalised slope.
5. Replace the accuracy proxy with precision, recall, F1 and false-alarm rate on labelled multi-event trials, and tune τ on a held-out set.
6. Complete the bibliography and re-check the literature matrix against full texts.

References

[1] Duty-cycling techniques for wireless sensor networks: A survey. Add exact authors, title, venue, and year from the survey used in the literature review.

[2] Energy-efficient data acquisition using least-mean-square adaptive prediction for wireless sensor networks. ResearchGate publication record. URL `https://www.researchgate.net/publication/220761399`. Office-temperature dataset study; authors, venue, and year should be verified.

[3] Forecasting-plus-threshold anomaly detection for resource-constrained spacecraft systems. Descriptive title based on the report's summary; add exact authors, venue, year, and identifier.

[4] Joint sensing and energy-harvesting task scheduling for wireless sensor networks. Add exact authors, title, venue, and year from the source used in the literature review.

[5] Telemetry system with predictive anomaly detection and pre-emptive high-rate sampling. Patent cited in the report; add patent number, inventors, assignee, and filing/publication date from the patent record.

[6] An energy-efficient adaptive sampling technique ensuring bounded data quality for wireless sensor networks. ResearchGate / publication record, 2013. URL `https://www.`

researchgate.net/publication/261273341. Author, venue, and year should be verified against the original publication.

[7] Edge machine learning: A survey, 2020. General descriptor based on the report; replace with the exact cited publication metadata.

[8] Forecasting-based protocol for reduced data transmission and extended battery life in livestock monitoring, 2021. Descriptive title based on the report's summary; exact publication metadata needs verification.

[9] Most efficient sensor network protocol for a permanent natural disaster monitoring system: Adaptive sampling with recoverable energy-harvesting strategy. arXiv preprint, 2021. URL https://arxiv.org/pdf/2103.11098. ASMP/RASA; verify authors and final publication details.

[10] Green edge artificial intelligence: Energy-efficient computing and sensing at the edge, 2023. General descriptor based on the report; replace with the exact cited survey or study.

[11] Duty-cycling and wake-up schemes exploiting correlated activity across devices, 2024. Descriptive title based on the report's summary; exact title and publication metadata need verification.

[12] Dependency-aware sampling period selection for adaptive sensing, 2025. Descriptive title based on the report's summary; replace with exact authors, title, venue, and DOI.

[13] Energy-aware edge computing and compute placement, 2025. General descriptor based on the report; replace with the exact cited publication metadata.

[14] Two-to-one event trigger for reducing co2 measurements on a microcontroller, 2025. Descriptive title based on the report's summary; replace with the exact paper title and complete authors/venue after verification.

[15] Batteryless iot devices under intermittent energy supply, 2026. Descriptive title based on the report's summary; exact source metadata needs verification.

[16] Edgespike: Spiking neural networks for low-power autonomous sensing in edge iot architectures, 2026. Verify author list, publication venue, DOI, and exact reported results against the original source.

[17] Low-power sensor node combining adaptive duty cycling with an on-device classifier, 2026. Report describes MATLAB/Simulink simulation; replace descriptive title with exact source metadata.

[18] Smart energy management in agricultural wireless sensor nodes using tinyml-based adaptive sampling. *Sensors*, 26(7):2014, 2026. doi: 10.3390/s26072014. URL https://doi.org/10.3390/s26072014. Verify author list against the publisher record.

[19] H. Algabroun and L. Håkansson. Parametric machine learning-based adaptive sampling algorithm for efficient iot data collection in environmental monitoring. *Journal of Network and Systems Management*, 33(5), 2025. doi: 10.1007/s10922-024-09881-1. URL https://doi.org/10.1007/s10922-024-09881-1.

[20] Cesare Alippi and Manuel Roveri. An adaptive sampling algorithm for effective energy management in wireless sensor networks with energy-hungry sensors. 2009. Bibliographic details should be checked against the exact snow-monitoring case study cited in the report.

[21] Analog Devices. *ADXL345: Three-Axis, ±2 g/±4 g/±8 g/±16 g Digital Accelerometer Data Sheet*. Analog Devices. URL `https://www.analog.com/media/en/technical-documentation/data-sheets/ADXL345.pdf`. Consult the data sheet revision used for the project.

[22] International Telecommunication Union. Itu-t recommendation l.1341: Energy efficiency measurement and metrics for telecommunication equipment. Technical report, ITU-T. URL `https://www.itu.int/rec/T-REC-L.1341`. Verify the recommendation title and edition against the exact L.1341 document cited in the report.

[23] OASIS. Mqtt version 3.1.1. OASIS Standard, 2014. URL `https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html`.

[24] Texas Instruments. *INA219 Zerø-Drift, Bidirectional Current/Power Monitor With I2C Interface*. Texas Instruments. URL `https://www.ti.com/lit/ds/symlink/ina219.pdf`. Consult the data sheet revision used for the project.

[25] Dimitrios Trihinas, George Pallis, and Marios D. Dikaiakos. Low-cost adaptive monitoring techniques for the internet of things: Exponential double smoothing-based adaptive sampling. EDSAS; referenced in the report through an adaptive-sampling study. URL `https://www.academia.edu/8679102/`. Confirm exact title, publication venue, and year from the original source.